

EE-5102/CS-6304 – Advanced Topics in Machine Learning

Assignment 0 – Introduction

31 July 2026

Guidelines

In Assignment 0 of ATML, you will focus on setting up the foundation for the rest of the course by brushing up your fundamental concepts about deep learning basics, preparing a reproducible workflow, reporting your work in a professional format, and demonstrating the ability to document your findings properly. Please note the following requirements carefully:

- Students are required to create a public GitHub repository and provide its link as part of the submission.
- Students must write a full report detailing their findings in the NeurIPS format provided.
- This assignment is strictly Pass/Fail. Please take your time to go through the corresponding material first before you jump to start the Tasks.
- It is highly recommended to code most of the stuff on your own; however, you are allowed to use GPT/Claude etc., for the coding part BUT you must have understanding of the code dynamics and you must be able to relate these dynamics with the internal working of the models.
- Individual submission is mandatory for all students.
- The deadline for submission is: **20 August 2026**.

1 CNNs – We will NOT cover this in class

CNNs remain the most important inductive-bias-heavy baseline for vision, where local convolutions exploit translation structure, hierarchical feature extraction supports strong transfer, and residual learning made very deep backbones trainable in practice. AlexNet established the large-scale deep CNN breakthrough, VGG demonstrated the value of depth with small kernels, and ResNet made deeper optimisation substantially easier through residual learning. Torchvision still exposes ResNet as a first-class pretrained family for transfer learning. Go through the following resources before you jump to the task.

1. Chapter 24: [Convolutional Neural Networks](#)
2. Chapters 10 and 11 of the UDL Book: [Understanding Deep Learning](#)
3. [PyTorch ResNet Documentation](#)
4. [PyTorch Transfer Learning Tutorial](#)
5. YouTube lectures:
 - (a) [MIT 6.S191: Convolutional Neural Networks](#)
 - (b) [Stanford CS231N: Deep Learning for Computer Vision](#)
 - (c) [DeepLearning.AI: Convolutional Neural Networks](#)

Task 1: Inner Workings of ResNet-152

1. **Baseline Setup**

- (a) Use a pre-trained ResNet-152 from PyTorch. ¹
- (b) Replace the final classification layer to match a smaller dataset such as CIFAR-10. ²
- (c) Train only the classification head while freezing the rest of the backbone. ³
- (d) Record training and validation performance for a few epochs.

Why is it unnecessary (and impractical) to train ResNet-152 from scratch on small datasets? What does freezing most of the network tell us about the transferability of features?

2. Residual Connections in Practice

- (a) Disable skip connections in a few selected residual blocks and re-train the modified network head. ⁴
- (b) Compare training dynamics and validation accuracy with the baseline.

How do skip connections change gradient flow in very deep networks? What happens to convergence speed and performance when residuals are removed?

3. Feature Hierarchies and Representations

- (a) Collect features from early, middle, and late layers of the network. ⁵
- (b) Visualize these features using dimensionality reduction (t-SNE or UMAP). ⁶

How does class separability evolve across layers? What differences can you observe between low-level and high-level representations?

4. Transfer Learning and Generalization

- (a) Fine-tune the model on a dataset different from ImageNet. ⁷
- (b) Compare performance between:
 - (i) using ImageNet-pretrained weights, and
 - (ii) training from random initialization.
- (c) Experiment with fine-tuning only the final block versus the full backbone.
- (d) Compare performance between:
 - (i) using ImageNet-pretrained weights, and
 - (ii) training from random initialization.
- (e) Experiment with fine-tuning only the final block versus the full backbone.

Which setting provides the best trade-off between compute and accuracy? Which layers seem most transferable across datasets, and why?

5. Optional Experiments

- (a) Compare t-SNE vs. UMAP in representing feature separability. ⁸
- (b) Analyze feature similarities between classes that ResNet tends to confuse.
- (c) Compare feature quality from ResNet-152 with a shallower ResNet (e.g., ResNet-18). ⁹

¹<https://pytorch.org/vision/stable/models/generated/torchvision.models.resnet152.html>

²https://pytorch.org/tutorials/beginner/finetuning_torchvision_models_tutorial.html

³https://pytorch.org/docs/stable/generated/torch.nn.Module.html#torch.nn.Module.requires_grad

⁴<https://arxiv.org/abs/1512.03385>

⁵https://pytorch.org/docs/stable/generated/torch.nn.Module.html#torch.nn.Module.register_forward_hook

⁶<https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>, <https://umap-learn.readthedocs.io/en/latest/>

⁷https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

⁸<https://distill.pub/2016/misread-tsne/>

⁹<https://pytorch.org/vision/stable/models/resnet.html>

2 Understanding Self-attention & Vision Transformers (ViT) – We will study this in class as well but very briefly

Self-attention replaced recurrence and convolutions in the original Transformer by letting each token attend to every other token through learned query-key-value interactions. Vision Transformer adapts that idea to images by splitting an image into patches treated as tokens, adding positional embeddings, and using a class token for classification; DeiT later showed that data-efficient ViT training is possible on ImageNet-scale data without massive external pretraining. Go through the following resources before you attempt the next task.

1. **Understanding Deep Learning**, available at: <https://udlbook.github.io/udlbook/>. See Chapter 12 for material on Transformers.
2. **The Illustrated Transformer**, available at: <https://jalammar.github.io/illustrated-transformer/>. This is a useful visual introduction to Transformer architectures.
3. **Exploring Visual Attention in Transformer Models**, available at: <https://medium.com/@nivonl/exploring-visual-attention-in-transformer-models-ab538c06083a>. This article discusses the generation and interpretation of attention maps.
4. **Hugging Face Model Outputs**, available at: https://huggingface.co/docs/transformers/en/main_classes/output. This documentation explains model outputs such as hidden states and attention tensors.
5. Dosovitskiy et al., *An Image Is Worth 16×16 Words: Transformers for Image Recognition at Scale*. The canonical ViT paper introducing image patches as tokens and class-token-based image classification.
6. Touvron et al., *Training Data-Efficient Image Transformers and Distillation Through Attention*. An important resource for practical ViT training without extremely large pretraining datasets.
7. **Torchvision VisionTransformer Documentation**, available at: https://docs.pytorch.org/vision/stable/models/vision_transformer.html. It provides official model builders and pretrained weights for the Torchvision ViT family.
8. **Hugging Face ViT Documentation**, available at: https://huggingface.co/docs/transformers/en/model_doc/vit. It provides API details for using pretrained ViT checkpoints and extracting hidden states and attention values.
9. **Google Research Vision Transformer Repository**, available at: https://github.com/google-research/vision_transformer. This repository contains official ViT code and checkpoints from the Google Research implementation.
10. **DINO Attention-Visualization Script**, available at: https://github.com/facebookresearch/dino/blob/main/visualize_attention.py. This is a practical reference for visualizing self-attention in vision models.
11. Recommended video lectures:
 - (a) **3Blue1Brown: Attention in Transformers, Step by Step**. A highly visual explanation of queries, keys, values, self-attention, and multi-head attention.
 - (b) **freeCodeCamp: Vision Transformer from Scratch Tutorial**. Covers patch embeddings, positional embeddings, self-attention, and implementation of the complete ViT architecture.
 - (c) **ExplainingAI: Image Classification Using Vision Transformer**. Includes ViT implementation, image classification, and visualization of attention maps using attention rollout.
 - (d) **MIT 6.5940: Lecture 16 – Vision Transformer**. A formal lecture on Vision Transformers by Prof. Song Han.

Task 2: Understanding ViT

1. Using a Pre-trained ViT¹⁰ for Image Classification:

Choose a small pre-trained ViT model from PyTorch, TIMM, or Hugging Face¹¹, which was pre-trained on

¹⁰<https://medium.com/@nivonl/exploring-visual-attention-in-transformer-models-ab538c06083a>

¹¹https://pytorch.org/vision/stable/models/vision_transformer.html, https://huggingface.co/docs/transformers/main_classes/output

ImageNet. Use the corresponding image processor (feature extractor) to prepare input images.

Select 1–3 images to test the model on. These could be from a standard dataset (e.g., an ImageNet sample) or any images of your choice. Ensure that they are appropriately sized, e.g., 224×224 pixels, or resize them using the feature extractor.

Run the images through the ViT model to obtain predicted class labels. Record the top-1 prediction—the model’s guess for the object in the image—and state whether the prediction seems reasonable.

2. Visualizing Patch Attention:

One advantage of transformers in vision is the ability to visualize attention maps over image patches. These maps can serve as a form of model interpretability, analogous to saliency maps in CNNs. Create an attention-based visualization for the ViT output by completing the following steps:

- Configure the model to output attention weights. In Hugging Face’s `ViTModel` or `ViTForImageClassification`, pass `output_attentions=True` when calling the model, similarly to the NLP case.
- Focus on the class token’s attention in the final layer. In ViTs, a special learnable [CLS] token is appended to the sequence of patch embeddings, and its output is used for classification.

The attention weights from the [CLS] token to all patch tokens in the final layer indicate which patches were most influential in the classification decision. Extract the attention matrix from the final transformer layer. It should have the shape: `(batch_size, num_heads, seq_len, seq_len)` where `seq_len =  $N_{\text{patches}} + 1$` and the additional token is the class token.

- Aggregate the attention heads for simplicity. For example, compute the average over all attention heads in the final layer, or use a single head if it appears to focus well.

Locate the row—or column, depending on the implementation—corresponding to the [CLS] token’s attention to the patch tokens. This produces a vector whose length equals the number of patches and whose values represent how much attention the class token assigned to each patch.

- Reshape the attention vector into the two-dimensional spatial arrangement of image patches. For an image of size 224×224 divided into 16×16 patches, the number of patches is

$$\left(\frac{224}{16}\right)^2 = 14 \times 14 = 196.$$

Therefore, the attention values can be reshaped into a 14×14 attention map.

- Visualize the attention map. Upsample the patch-level attention map to the original image resolution and overlay it on the image to identify the highlighted regions.

This is similar to creating a heat-map overlay. Use a library such as Matplotlib or PIL to create a semi-transparent red overlay whose intensity corresponds to the attention weight.

3. Analyze the Attention Map:

Include the image containing the attention-map overlay and describe what you observe. Did the ViT focus on image regions corresponding to the predicted object class?

For example, if the image contains a dog and the model predicts “dog,” determine whether the highlighted patches outline the dog’s figure. This provides insight into whether the model is using appropriate visual features or relying on the background or other contextual cues.

Conceptually compare the attention-based explanation with typical CNN interpretation methods such as CAM or Grad-CAM. You do not need to implement CAM or Grad-CAM. Discuss the potential advantages of transformers having built-in attention mechanisms for interpretability.

Note any unusual behavior, such as the model attending to an irrelevant region. Are different attention heads specialized? Explain how you can determine whether specialization is occurring.

4. Patch-Masking Experiment:

Mask a fraction of the input patches during inference and observe the effect on classification accuracy. How robust is the ViT to missing patches, and why?

Compare random patch masking with structured masking, such as masking patches in the center of the image. Describe and explain the differences you observe.

5. CLS Token vs. Mean-Pooled Patch Tokens:

Compare linear probes trained using the [CLS] token with linear probes trained using the mean of all patch tokens.

Which pooling method performs better, and why? How might the choice of pooling method interact with different pre-training objectives?

3 Contrastive Learning and CLIP Model – We will study this in class as well but very briefly

Motivation: Mutual Information and Contrastive Learning: The journey of contrastive learning begins with the InfoMax principle – the idea that a good representation of data should preserve as much mutual information between the representation and the input as possible. Recall: mutual information (MI) between two random variables X and Y is defined as:

$$I(X; Y) = H(X) - H(X | Y) = H(Y) - H(Y | X) = \mathbb{E}_{p(x,y)} \left[\log \frac{p(x,y)}{p(x)p(y)} \right]$$

Intuitively, $I(X; Y)$ measures how much knowing one of the variables reduces uncertainty about the other. Maximizing $I(X; Y)$ means learning a representation $Y = f(X)$ that retains all relevant information about X . Early research (e.g. Deep InfoMax, Hjelm et al. 2019) advocated training encoders by directly maximizing an estimate of $I(X; Z)$ between inputs X and learned embeddings Z .

However, computing and maximizing MI directly is challenging – it requires estimating high-dimensional densities.

Contrastive learning provides a practical solution by turning representation learning into a classification problem: distinguishing between positive pairs (related samples likely to have high MI) and negative pairs (unrelated samples). This approach was inspired by techniques like Noise-Contrastive Estimation (NCE), which train models to tell apart real data from noise. In representation learning, we designate pairs of outputs as positive if they come from the same underlying input (or context) and negative if they do not, and train an encoder to score positives higher than negatives. By doing so, the encoder implicitly maximizes mutual information between representations of positive pairs.

A landmark formulation is the InfoNCE loss (introduced by van den Oord et al., 2018 in Contrastive Predictive Coding). Suppose we have an anchor context $\mathbf{c}$ and a set $X = \{\mathbf{x}_{\text{pos}}, \mathbf{x}_1^-, \dots, \mathbf{x}_{N-1}^-\}$ containing one positive sample $\mathbf{x}_{\text{pos}}$ (truly associated with $\mathbf{c}$) and $N - 1$ negatives (drawn independently). We train a scoring function $f(\mathbf{x}, \mathbf{c})$ (often f is an exponential of a similarity score) to identify the positive. The probability of picking the positive out of N is:

$$p(\text{pos} | \mathbf{c}, X) = \frac{f(\mathbf{x}_{\text{pos}}, \mathbf{c})}{\sum_{j=1}^N f(\mathbf{x}_j, \mathbf{c})}.$$

The InfoNCE loss is the negative log-likelihood of the positive, averaged over samples:

$$\mathcal{L}_{\text{InfoNCE}} = -\mathbb{E}_{X, \mathbf{c}} \left[\log \frac{f(\mathbf{x}_{\text{pos}}, \mathbf{c})}{\sum_{j=1}^N f(\mathbf{x}_j, \mathbf{c})} \right].$$

Crucially, minimizing $\mathcal{L}_{\text{InfoNCE}}$ maximizes a lower bound on mutual information. In fact, it can be shown that:

$$I(\mathbf{x}; \mathbf{c}) \geq \log N - \mathcal{L}_{\text{InfoNCE}}.$$

This inequality (proved in CPC’s analysis) means that as the contrastive classifier gets better (InfoNCE loss decreases), the mutual information between $\mathbf{x}$ and its context $\mathbf{c}$ increases. In other words, the contrastive objective is aligned with InfoMax. This insight – treating representation learning as maximizing MI via contrastive classification – guided the emergence of contrastive learning as a paradigm. Rather than require manual labels, the structure in data itself (what co-occurs with what) can provide a supervision signal to encode information.

In short: Mutual information provides the theoretical motivation: a representation that maximizes MI with the input is highly informative. Contrastive learning operationalizes this by creating training tasks where the model must identify related pairs among noise – effectively teaching the model to preserve information that distinguishes inputs. This approach has become central to self-supervised learning, as we explore next.

3.1 Learning Without Labels: Unsupervised Contrastive Learning:

A major challenge in modern machine learning is leveraging unlabeled data. Contrastive learning has proven to be a powerful approach for self-supervised representation learning when explicit labels are unavailable. The core idea

is to generate “pseudo-labels” from the data itself by defining what constitutes a similar pair (positive) versus a dissimilar pair (negative). The most common technique is to use data augmentation: apply two different random transformations to the same input, producing two views that are treated as a positive pair. Because these two views originate from the same source, the model should make their embeddings agree. Meanwhile, other samples in the batch (augmented from different source images) serve as negatives that the model should push apart in the embedding space. This instance discrimination framework effectively treats each image as its own class, with augmentations not changing the identity.

Key ingredients for successful unsupervised contrastive learning, as identified by Chen et al. (SimCLR) and others, include:

- heavy and diverse data augmentations,
- a large number of negatives per positive (which often implies large batch sizes or memory banks), and
- an appropriate embedding normalization and similarity measure (typically cosine similarity with temperature).

Let’s review some seminal frameworks and how each innovation improved unsupervised contrastive learning:

1. SimCLR (2020) – A Simple Framework for Contrastive Learning of Visual Representations

By Chen et al. established a straightforward recipe: take an image, create two random augmented versions, encode them, and use a contrastive loss to make their representations similar and all others in the batch dissimilar. SimCLR introduced a normalized temperature-scaled cross-entropy loss (NT-Xent), which is essentially the InfoNCE loss applied within a training batch. For an anchor i and its positive j , SimCLR’s loss is:

$$\mathcal{L}_{\text{SimCLR}}(i, j) = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)},$$

where $2N$ is the batch size (each of N original images yields 2 augmented instances), and $\text{sim}(\cdot, \cdot)$ is cosine similarity. Importantly, SimCLR added a nonlinear projection head $g(h)$ that maps the encoder’s representation h to a latent vector z for the contrastive loss – this was found to improve performance since the contrastive objectives might remove information (the head g can discard non-essential features) while the encoder f still retains useful features in h . SimCLR’s recipe (strong augmentations, large batches, a projection MLP, and cosine similarity with temperature) yielded greatly improved representations on ImageNet. However, it required very large batch sizes (e.g., batch 4096) to work well, because all negatives came from within the batch. With smaller batches, the diversity of negatives was insufficient and performance suffered. This reliance on big batches (hence big memory and TPU/GPU compute) was a limitation.

2. MoCo (Momentum Contrast, 2020)

Kaiming He et al. tackled the batch size issue by introducing a memory queue of features and a momentum-updated encoder. Instead of comparing only within a single batch, MoCo maintains a dictionary (queue) of key representations from recent batches, yielding a large set of negatives without requiring a huge current batch. MoCo uses two networks: a query encoder (updated by backprop) and a key encoder (updated slowly via momentum to maintain consistency of representations in the queue). An incoming batch’s augmented samples are encoded by the query encoder for the anchor, and by the key encoder for the positive, and compared against a queue of past keys as negatives. The loss is again InfoNCE-based:

$$\mathcal{L}_{\text{MoCo}} = -\log \frac{\exp(\mathbf{q} \cdot \mathbf{k}^+/\tau)}{\exp(\mathbf{q} \cdot \mathbf{k}^+/\tau) + \sum_{i=1}^K \exp(\mathbf{q} \cdot \mathbf{k}_i^-/\tau)},$$

where $\mathbf{q}$ is the query embedding, $\mathbf{k}^+$ is the key for the matching image, and $\mathbf{k}_i^-$ are keys from the queue. By decoupling negatives from the batch, MoCo allowed using a small batch size while still having a large effective number of negatives (the queue can be thousands deep). This made contrastive training more memory-efficient. MoCo v2 (Chen et al. 2020) further incorporated SimCLR’s tricks like the MLP projection head and stronger augmentations, closing the gap to SimCLR’s accuracy without giant batches.

3. BYOL (Bootstrap Your Own Latent, 2020)

Grill et al. took a bold step: can we learn good representations without any negative pairs at all? BYOL demonstrated that it is possible to avoid explicit negatives by using a form of self-distillation. BYOL employs two neural networks: an online network and a target network, both with the same architecture (e.g., a ResNet encoder and a projection MLP), but the target network’s parameters are an exponential moving average of the

online network’s parameters. Training proceeds as follows: given an image, generate two augmented views v and v' . The online network (parameters θ) produces an embedding $y_\theta = f_\theta(v)$ and projection $z_\theta = g_\theta(y_\theta)$. The target network (parameters ξ) produces $y'_\xi = f_\xi(v')$ and $z'_\xi = g_\xi(y'_\xi)$. Then the online network predicts the target projection: $p_\theta = q_\theta(z_\theta)$, where q is an additional predictor MLP. The BYOL loss is simply the mean squared error between the normalized predictions and the normalized target projections:

$$\mathcal{L}_{\text{BYOL}} = \|\bar{q}_\theta(z_\theta) - \bar{z}'_\xi\|_2^2,$$

where $\bar{q}$ and $\bar{z}'$ denote L^2 -normalized vectors. This loss is symmetrized (we compute it for both v predicting v' and vice versa). BYOL’s surprising finding was that even without negatives, the representation $f_\theta(x)$ doesn’t collapse to a trivial constant – as long as we have the target network and predictor in place. The stop-gradient on the target network (it is not directly optimized, only updated via momentum) is critical to prevent collapse, as is the use of batch normalization in the encoder.

In essence, BYOL works like a teacher-student paradigm: the student (online) learns to predict the teacher (target) representation of an augmented view, and the teacher slowly updates to the student’s weights. This bootstrapping leads to learning useful features, achieving state-of-the-art results on ImageNet without any negative pairs. BYOL showed that explicit contrast between different samples isn’t the only way – you can learn via contrast with a previous self. This paved the way for other non-contrastive self-supervised methods (SimSiam, Barlow Twins, VICReg, etc.), though strictly BYOL’s use of two views and a slow-moving target is still in the contrastive family in a broader sense (contrastive methods seek to avoid collapse – BYOL achieves it implicitly).

How each innovation built on the last: SimCLR demonstrated the power of contrastive learning but required brute-force (big batches). MoCo decoupled negatives from batch size using a queue and momentum, making contrastive learning more practical resource-wise. BYOL challenged the necessity of negatives altogether, simplifying the framework and revealing that representational collapse can be averted through architectural choices even without an explicit repulsion term. Each of these developments led to advancements in unsupervised learning: models like SimCLR, MoCo, BYOL (and their variants) produced image representations that rival supervised ImageNet features for transfer learning tasks. The success of these methods also stimulated theoretical work into why they work (e.g., the role of implicit negatives or regularization) and practical adoption in many domains (not just vision, but text, audio, graphs, etc., where similar contrastive approaches like CPC, SimCLR analogues, have been applied).

In short: contrastive learning without labels leverages the inherent structure in data via augmentations and pairwise comparisons. By carefully constructing training signals (positives and negatives), models learn to encode invariances (same object under view changes) and discard spurious variations – all without any human-provided labels. This forms a foundation that we next extend to scenarios with labels.

3.2 Contrastive Learning with Available Labels:

When class labels or other annotations are available, contrastive learning can still be extremely useful. In fact, integrating label information into contrastive objectives often yields better feature learning than plain classification losses. We consider two scenarios: (a) using class labels alone to form positive/negative pairs, and (b) using multimodal pairings (e.g. images paired with text captions). In both cases, the core idea is to use contrastive losses to pull semantically related samples together in the embedding space and push unrelated ones apart. Classic metric learning sought to learn embeddings where items of the same class are close and different classes are far apart. Contrastive losses with labels were studied long before the current self-supervised wave. Notable examples include:

Triplet Loss (Hadsell et al. 2006; Schroff et al. 2015): This loss operates on triplets of samples: an anchor x , a positive x^+ (same class as anchor), and a negative x^- (different class). The goal is to ensure the anchor is closer to the positive than to the negative by a margin. A common form:

$$\mathcal{L}_{\text{triplet}} = \max \{0, d(f(x), f(x^+)) - d(f(x), f(x^-)) + \alpha\},$$

where $d(\cdot, \cdot)$ is a distance (e.g., Euclidean) and α is a margin. The loss is zero if the positive is sufficiently closer than the negative by at least α ; otherwise it penalizes the difference. Triplet loss was popular for tasks like face recognition (FaceNet) and fine-grained image retrieval, but it requires careful negative mining – choosing hard but

true negatives to form informative triplets. Random negatives are often too easy (most different-class pairs are already far apart), so techniques like semi-hard negative mining were developed.

N-pair Loss (Sohn 2016): This is a multi-class extension of triplet loss that considers multiple negatives at once, akin to a classification loss. In an N-pair setup, we take one anchor and one positive of the same class, plus $N - 1$ negatives of different classes in a tuple. The loss is derived from a softmax that the anchor should be closest to its positive among all N options. A typical formulation is:

$$\mathcal{L}_{\text{N-pair}}(x, x^+, \{x_i^-\}) = -\log \frac{\exp(f(x)^\top f(x^+))}{\exp(f(x)^\top f(x^+)) + \sum_{i=1}^{N-1} \exp(f(x)^\top f(x_i^-))}.$$

This looks very much like the InfoNCE / contrastive loss, and indeed, N-pair loss and InfoNCE are mathematically closely related – both are cross-entropy losses over multiple negatives. N-pair loss can be seen as a supervised version (using class negatives rather than random augmentation negatives). If only one negative is used ($N = 2$ total), N-pair reduces to a form similar to a triplet loss. With many negatives, it encourages separation of the positive from all other classes’ examples in the batch.

Supervised Contrastive Learning (SupCon, Khosla et al. 2020): Recent work explicitly combined the strengths of self-supervised contrastive learning with label information. In SupCon, instead of only using one positive per anchor, all other samples of the same class in the batch are treated as positives for the anchor. This generalizes the contrastive loss to a one-vs-many setting using labels. In practice, the SupCon loss is written as:

$$\mathcal{L}_{\text{sup}} = -\frac{1}{|\mathcal{P}(i)|} \sum_{p \in \mathcal{P}(i)} \log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_p)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)},$$

where $\mathcal{P}(i)$ is the set of indices in the batch with the same class as sample i (positives). Essentially, for each anchor i , you average the InfoNCE-style loss over all positives p of the same class in the batch. This encourages intra-class clustering (all embeddings of the same class pulled together) and inter-class repulsion. Khosla et al. showed that supervised contrastive pre-training followed by a simple linear classifier can slightly outperform a standard cross-entropy classifier on ImageNet, and is more robust to noisy labels and other perturbations. SupCon can be seen as a blend of contrastive learning and classical supervised learning: it uses label groups to define similarity, but does not explicitly train a classifier during representation learning.

These losses leverage labels to define what “similar” means, rather than just using augmentations of the same instance. In effect, they learn a representation space with explicit class separation. Notably, if the model perfectly separates classes in feature space, a simple k -NN or linear classifier will do well – that’s the aim of these methods. They have been applied in tasks like metric learning (e.g., product image retrieval, face identification) and as a regularizer for classification. In practice, implementing supervised contrastive losses often requires careful batching (to ensure enough positives of each class per batch) or sampling strategies (like ensuring each class appears multiple times per batch).

Comparing supervised contrastive vs. cross-entropy: Cross-entropy with softmax classification indirectly pushes classes apart, but focuses on decision boundaries and can suffer from issues like label noise sensitivity. A contrastive approach with labels enforces a stronger criterion: not only should each sample be classified correctly, but all same-class samples should be grouped in the embedding space. This can improve robustness and representation quality. However, it may require more careful hyperparameters (temperature tuning, etc.) and large batches (for many positives), similar to self-supervised contrastive learning.

3.3 Multimodal Contrastive Learning – The Case of CLIP:

Contrastive learning becomes especially powerful when dealing with multimodal data, where the “labels” are implicit in paired examples from different modalities. A prime example is learning from image–text pairs: we might not have class labels for images, but we often have a caption or description for an image. Such pairs provide cross-modal supervision – the image and its accompanying text caption are related (positive pair), whereas an image with a random different caption is unrelated (negative). CLIP (Contrastive Language–Image Pre-Training) by Radford et al. (2021) is a seminal model that uses contrastive learning in this multimodal context.

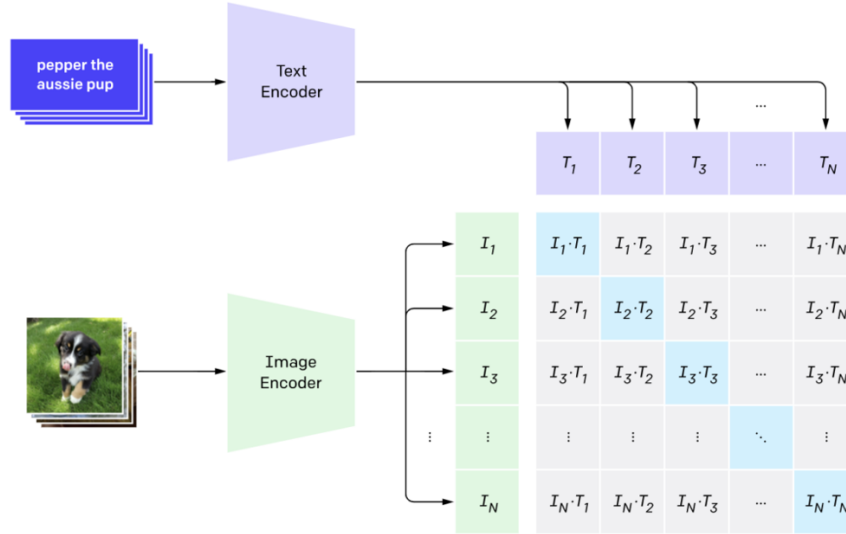


Figure 1: **Illustration of CLIP’s dual-encoder approach:** a batch of texts (captions) and images are encoded separately. A similarity matrix is computed between all image embeddings I_i (rows) and text embeddings T_j (columns). CLIP’s contrastive loss encourages the diagonal elements (correct I_i – T_i pairs, highlighted in blue) to be large, while off-diagonals are pushed lower

CLIP’s Architecture: CLIP consists of two separate encoders – one for images and one for text – that produce embeddings in a common vector space. For example, CLIP’s ViT-B/32 image encoder and Transformer text encoder each output a 512-dimensional feature vector for an image or a text input, respectively. The encoders are trained jointly so that matching image-text pairs have higher similarity (dot product or cosine) than non-matching pairs. The training objective is essentially an InfoNCE loss that treats the correct pairing as positive and all other pairings in the batch as negatives. In practice, CLIP uses a symmetric loss: it maximizes the cosine similarity of the correct image-caption pairs while minimizing it for incorrect pairings, from both the image-to-text and text-to-image directions (two cross-entropy losses are averaged). This trains the image and text encoders to produce aligned representations.

CLIP’s Training Regime: What made CLIP especially impactful was the scale and diversity of training data. It was trained on roughly 400 million (image, text) pairs gathered from the internet. These pairs cover an extremely broad range of visual concepts and descriptions in natural language. Unlike traditional supervised learning which might classify images into a fixed 1000 categories, CLIP’s “labels” are free-form text descriptions – effectively an open vocabulary. By training on this, CLIP learned a very general visual representation space tied to language semantics. The optimization used large batch sizes (e.g., 32k) to accommodate many negatives (since each image in a batch serves as a negative for the other images’ captions, and vice versa). The result was a model that could be applied zero-shot to many tasks: you can feed an image and some text labels and find which label’s embedding is closest to the image’s embedding – performing image classification without any training on that specific task.

How CLIP changed the landscape: CLIP’s release demonstrated for the first time that a single pretrained model could achieve robust zero-shot image classification on datasets like ImageNet, just by providing the class names as text prompts. For instance, to classify an image, one can compute CLIP’s similarity between the image and the text “a photo of a cat”, “a photo of a dog”, etc., and pick the highest – effectively doing classification via language descriptors. CLIP often matched or exceeded the accuracy of fully supervised models on many datasets without any fine-tuning. Beyond classification, CLIP’s embeddings turned out to be useful for a slew of tasks: image-text retrieval (finding the caption that matches an image or vice versa), evaluating image generation results (by checking alignment between generated image and prompt), and as a general backbone for multimodal tasks. The model has a strong grasp of visual semantics grounded in language – it knows that “pepper the Aussie pup” might refer to a puppy image (as in the figure above), and can relate images to textual concepts remarkably well.

Several concurrent and subsequent works followed similar approaches (e.g., Google’s ALIGN was trained on an even larger private dataset of image-text pairs). The success of CLIP confirmed that natural language is a powerful source of supervision for visual learning. Essentially, CLIP leveraged captions as labels, but on an unprecedented scale and with a contrastive objective that forces joint embedding learning.

3.4 CLIP’s Impact on Vision-Language and Beyond:

The introduction of CLIP has had a profound impact on downstream deep learning tasks, especially those involving vision and language. By providing a powerful pre-trained vision-language model, CLIP has become a building block in numerous applications:

- **Vision-Language Models (VLMs) and Multimodal Understanding:** CLIP-type models enable systems that jointly reason about images and text. Tasks like zero-shot image classification, image captioning, visual question answering, and cross-modal retrieval have all benefited. For example, CLIP can be used to encode both an image and a question about the image, and then a separate language model can generate an answer – the strong image encoding improves the overall system. Many open-world recognition problems (where the set of categories is not fixed) now rely on CLIP. Instead of training a classifier for new categories, one can often use CLIP in a zero-shot manner or fine-tune it lightly. The CLIP paper showed significant gains in robustness and stress tests for classification – models like CLIP are far less confused by unusual images or distributions because of their broad training. Additionally, CLIP’s embeddings have been used for semantic search (find images in a database that match a text query, or vice versa) with great success, far outperforming traditional keyword or hand-engineered feature searches. Essentially, CLIP provided a universal feature space for vision and language, which researchers have readily adopted for many tasks that require understanding visual content and natural language together.
- **Enabling Zero-Shot and Few-Shot Learning:** Before CLIP, computer vision models typically required fine-tuning for each new task or category. CLIP changed that paradigm by showing broad zero-shot ability. For instance, one can tackle a new classification task by simply writing appropriate prompt texts for each category (e.g., “a photo of a [classname]”) and using CLIP’s similarities to classify images – no further training needed. This has huge practical importance: models can be deployed to recognize new concepts without collecting new labeled datasets, as long as you can describe the concept in words. This idea has been extended in various ways, such as prompt engineering or prompt learning for CLIP (where small textual prompts are optimized to improve performance on a target dataset without updating the image encoder). The ripple effect is that language as the universal label space has become a core theme in multimodal AI.
- **Foundation for Generative Models (Diffusion and beyond):** Perhaps one of the most exciting impacts of CLIP is on text-to-image generation. Models like DALL·E 2 and Stable Diffusion build upon the idea of CLIP. In fact, DALL·E 2 uses CLIP in a two-step process: first a CLIP text encoder maps a prompt to an embedding; then a diffusion model generates an image conditioned on that embedding (there’s also a CLIP image decoder component in DALL·E 2’s pipeline). Stable Diffusion, a widely used open-source text-to-image diffusion model, uses OpenAI’s CLIP ViT-L/14 text encoder to condition the diffusion process on textual prompts. During training of these generative models, CLIP’s embedding space serves as the communication channel between text and image: the model learns to produce images whose CLIP image embeddings match the CLIP text embedding of the prompt. As a result, the quality and interpretability of generated images have dramatically improved – the generator benefits from CLIP’s rich cross-modal understanding.
- Moreover, CLIP itself has been used directly to guide generation. Before diffusion models took off, a line of work used CLIP as a loss function or feedback signal to iteratively optimize an image to match a text description (examples include CLIP-Guided VQGAN, CLIPDraw, and others). In such setups, CLIP evaluates how well an image fits a prompt (via cosine similarity in CLIP space), and this guides the image generation process. This concept of CLIP guidance remains in diffusion model sampling as well: one can modify the diffusion sampling to maximize CLIP score for a prompt, further improving alignment between generated images and the text. Indeed, CLIP is “the basis for many text-to-image models (e.g., Stable Diffusion) to calculate the distance between the generated image and the prompt during training”. In short, CLIP has become an integral part of the text-to-image revolution.
- With CLIP’s emergence, researchers even started to use CLIP to measure semantic accuracy of generated images or image captions. For example, CLIPScore is a metric that uses CLIP similarity between a generated caption and an image to judge caption quality, outperforming some traditional metrics. Similarly, to evaluate image outputs, one can compute how well the output image and the input text prompt align in CLIP space. This showcases how a strong contrastive model can serve as a kind of “universal judge” of cross-modal alignment.
- **Fine-tuning and Adaptation:** CLIP’s models (open-sourced by OpenAI and further reproduced by the community, e.g., OpenCLIP) have been fine-tuned for myriad downstream applications. For instance, by fine-tuning CLIP on a smaller labeled dataset, one can get a state-of-the-art classifier with far fewer labels than training from scratch. There have been successes in domains like medical imaging (using radiology report text and images to adapt CLIP), remote sensing, robotics (using CLIP’s image understanding to help language-based commands), and more. In multimodal research, CLIP is often the starting point – many later models use

CLIP’s encoders to then fuse image and text (e.g., by cross-attention) for tasks that require deeper interaction between modalities (like VQA). The availability of CLIP’s trained encoders has essentially raised the floor for vision-language research: anyone can leverage these representations instead of training from zero.

Reading List: Before you start attempting the Tasks, make sure you go through the following reading list with reasonable level of understanding:

1. InfoMax, Mutual Information & Unsupervised Contrastive Learning:

- Aaron van den Oord et al., “Representation Learning with Contrastive Predictive Coding,” arXiv preprint arXiv:1807.03748, 2018. The CPC paper that introduced InfoNCE and demonstrated powerful unsupervised learning by predicting future information in high-dimensional signals. <https://arxiv.org/abs/1807.03748>
- R. Hjelm et al., “Learning Deep Representations by Mutual Information Estimation and Maximization (Deep InfoMax),” ICLR 2019. This one Proposed directly maximizing mutual information in neural networks; provides InfoMax motivation and connections to contrastive learning. <https://arxiv.org/abs/1808.06670>
- SimCLR: Ting Chen et al., “A Simple Framework for Contrastive Learning of Visual Representations,” ICML 2020. The phenomenal SimCLR paper; introduced NT-Xent loss, importance of data augmentation, large batch training, and a projection head for contrastive SSL. <https://arxiv.org/abs/2002.05709>
- MoCo: Kaiming He et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” CVPR 2020. The MoCo v1 paper; uses a momentum encoder and queue for building a dynamic dictionary of negatives, followed by MoCo v2 (2020) which added improvements. <https://arxiv.org/abs/1911.05722>
- BYOL: Jean-Bastien Grill et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” NeurIPS 2020. Introduces BYOL’s online-target network framework with no negatives; a milestone in self-supervised learning, showing competitive results without contrastive pairs. There is also a blog post by author: “BYOL: Bootstrapping Your Own Latent”. <https://arxiv.org/abs/2006.07733>

2. Supervised Contrastive:

- Prannay Khosla et al., “Supervised Contrastive Learning,” NeurIPS 2020. This paper extended contrastive learning to labeled data by using label-based grouping, SupCon loss, and shows improved robustness over cross-entropy. <https://arxiv.org/abs/2004.11362>

3. CLIP:

- Alec Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” ICML 2021 (CLIP paper). Key paper showing image-text contrastive learning at scale; includes zero-shot experiments and robustness findings. <https://arxiv.org/abs/2103.00020>
- OpenAI Blog: <https://openai.com/index/clip/>

4. Blogs and Tutorials:

- (a) Lilian Weng’s blog “Contrastive Representation Learning” (2021) – an excellent high-level overview of contrastive learning ingredients and methods. <https://lilianweng.github.io/posts/2021-05-31-contrastive/>
- (b) Unit8’s blog “The Beginner’s Guide to Contrastive Learning” (v7labs, 2020) – explains SimCLR and contrasts with older methods in simple terms. <https://www.v7labs.com/blog/contrastive-learning-guide>
- (c) Hugging Face Course Chapter on CLIP or multimodal models – contains code examples of using CLIP models. <https://huggingface.co/learn/computer-vision-course/en/unit4/multimodal-models/tasks-models-part1>

5. Code Repositories:

- (a) lightly (GitHub: [lightly-ai/lightly](https://github.com/lightly-ai/lightly)) – A library with convenient pipelines for self-supervised learning (SimCLR, MoCo, BYOL implementations) and batch memory management. <https://github.com/lightly-ai/lightly>

- (b) PyTorch Metric Learning (GitHub: KevinMusgrave/pytorch-metric-learning) – Implements many loss functions (triplet, N-pair, contrastive) and miners for negatives; great for supervised embedding learning tasks. <https://kevinmusgrave.github.io/pytorch-metric-learning/losses/>
- (c) OpenCLIP (GitHub: LAION-AI/OpenCLIP) – Open reproduction of CLIP with trained models, helpful if you want to explore CLIP variants or train on your own data. https://github.com/mlfoundations/open_clip

Task 3: CLIP Tasks

1. Zero-Shot Classification on STL-10

- (a) Download the STL-10¹² dataset from Torchvision.
- (b) Load OpenAI’s CLIP model¹³ from the official CLIP implementation.
- (c) Evaluate CLIP for zero-shot classification accuracy on STL-10 using the following prompting techniques:
 - i. Plain labels, e.g., “cat”.
 - ii. Prompted text, e.g., “a photo of a cat”.
 - iii. More descriptive variants of the prompts.
- (d) You may experiment with the prompts on a small subset. However, compare the accuracies of at least three different prompting strategies on the complete test set.

2. Exploring the Modality Gap

- (a) Use the vision and text encoders within CLIP to extract image and label embeddings for 50–100 STL-10 samples.
- (b) Use a dimensionality-reduction technique such as UMAP or t-SNE to project the embeddings into a two-dimensional space.
- (c) Visualize and compare the distributions of the text and image embeddings.
- (d) Briefly explain your findings:
 - How separated are the two modalities?
 - Does normalization affect the modality gap?
 - Why does CLIP still perform well despite this gap?

3. Bridging the Modality Gap

- (a) One simple method for aligning the modalities is the orthogonal Procrustes transform. Given two sets of embeddings X , representing image features, and Y , representing text features, the goal is to find an orthogonal matrix R that minimizes

$$\min_R \|XR - Y\|_F,$$

where $\|\cdot\|_F$ is the Frobenius norm. The closed-form solution involves singular value decomposition (SVD), but a library implementation is sufficient for this task.

- (b) Pair the STL-10 image embeddings with their corresponding text embeddings.
- (c) Learn the optimal rotation matrix R using a library implementation of Procrustes alignment, such as `scipy.linalg.orthogonal_procrustes`, or an equivalent implementation.
- (d) Apply the rotation transform to the CLIP embeddings.

¹²<https://docs.pytorch.org/vision/main/generated/torchvision.datasets.STL10.html>

¹³<https://github.com/openai/CLIP>

- (e) Visualize the aligned embeddings using t-SNE or UMAP. How does the alignment affect the modality gap?
- (f) Recompute the classification accuracy using the aligned embeddings and compare the results with Part 0.

4 Variational Autoencoders — We will study this in class as well but very briefly

Deep generative modeling is a fundamental area of machine learning focused on learning the underlying distribution of data so that new, realistic samples can be generated. This field combines probability theory, deep learning, and variational inference to model complex, high-dimensional data distributions such as natural images, audio, or text. At its core, generative modeling learns a probability distribution $p(\mathbf{x})$ or $p(\mathbf{x} | \mathbf{z})$, where: $\mathbf{x}$ is the observed data (e.g., images, text), $\mathbf{z}$ is a latent (hidden) variable, typically drawn from a simple prior (e.g., standard Gaussian). The goal is to either:

- Explicitly model the distribution $p(\mathbf{x})$ or $p(\mathbf{x} | \mathbf{z})$ (e.g., via maximum likelihood), or
- Implicitly learn to sample from it, even if we don't know the exact form of the distribution.

Complete field is dominated by three paradigms:

- **Latent variable models:** introduce $\mathbf{z}$ and marginalize: $p(\mathbf{x}) = \int p(\mathbf{x} | \mathbf{z})p(\mathbf{z}) d\mathbf{z}$
- **Autoregressive models:** directly factorize the joint: $p(\mathbf{x}) = \prod_{i=1}^N p(x_i | x_{<i})$
- **Implicit models:** do not specify an explicit form for the data distribution $p(x)$ or $p(x | z)$, and instead focus on generating samples that match the data distribution. These models bypass likelihood estimation entirely and rely on comparing generated samples with real data via alternative objectives. A typical example is Generative Adversarial Networks (GANs), which define a generator network $G(z)$ that maps noise $z \sim p(z)$ to samples $x \sim q(x)$, and a discriminator that tries to distinguish real from generated data. Training proceeds as a two-player minimax game, driving the generated distribution $q(x)$ closer to the real data $p(x)$. Because no likelihood is used, evaluation and inference can be challenging, but sample quality is often very high. GANs learn direct mappings from noise to data in a single step: $z \rightarrow x$. Another example here is diffusion models which learn to iteratively denoise from pure noise to a data sample over many steps:

$$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0 \approx \text{data}.$$

Unlike GANs, diffusion models often have a well-defined training objective, such as minimizing the variational bound on negative log-likelihood or a denoising score-matching loss.

Deep Generative Models		
Latent Variable Models (VAE, Flow)	Autoregressive (PixelCNN, GPT)	Implicit Models (GAN, Diffusion)
Encodes $z \rightarrow x$	Factorizes x directly	Samples x via T
Used for latent space learning	Great for text and speech data	High-quality image synthesis

Reading List:

Before you start attempting the VAE Tasks, make sure you go through the following reading list with reasonable level of understanding:

1. Kingma, Diederik P., and Max Welling. "An introduction to variational autoencoders." Foundations and Trends® in Machine Learning 12.4 (2019): 307-392., available at: <https://arxiv.org/abs/1906.02691>. I have taken most of the content from its first two chapters.
2. Doersch, Carl. "Tutorial on variational autoencoders." arXiv preprint arXiv:1606.05908 (2016)., available at: <https://arxiv.org/abs/1606.05908>. This is an excellent resource which focuses on intuitive understanding.
3. The code repository by Doersch: https://github.com/cdoersch/vae_tutorial is a useful reference for the implementation parts, and his tutorial text provides insights on design choices and common tricks (such as using a Bernoulli output for pixel data and the significance of the KL term).
4. Chan, Stanley. "Tutorial on diffusion models for imaging and vision." Foundations and Trends® in Computer Graphics and Vision 16.4 (2024): 322-471., available at: <https://arxiv.org/abs/2403.18103>. Its Chapter 1 will give you deep understanding of VAEs along with math and how to implement it.
5. A very good blog on VAEs model: <https://medium.com/h7w/an-introduction-to-variational-autoencoders-vaes-305bacb75fbb>

Task 4: VAE Task

1. **Building a Basic VAE in PyTorch:** Implement a Variational Autoencoder for the MNIST dataset. Use PyTorch and design your model as follows:

- **Encoder:** The encoder network takes a 28×28 input image x and outputs parameters for the approximate posterior $q_\phi(z | x)$. Specifically, have the encoder produce a mean $\mu_\phi(x)$ and log-variance $\log \sigma_\phi^2(x)$ for a Gaussian latent variable z . You can use a multi-layer perceptron (MLP) or a small convolutional network. *Hint:* A simple MLP with one or two hidden layers (e.g., 400 or 500 neurons) and ReLU activations works well.
- **Reparameterization Trick:** Implement the reparameterization trick to sample $z \sim q_\phi(z | x)$. That is, sample $\epsilon \sim \mathcal{N}(0, I)$ and compute

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon.$$

PyTorch tip: you can use `torch.randn_like(...)` for ϵ , or use `torch.distributions.Normal` to construct the distribution.

- **Decoder:** The decoder network takes the sampled latent z and produces parameters for $p_\theta(x | z)$. For MNIST which consists of images in $[0, 1]$, assume a Bernoulli output distribution for each pixel. The decoder can output the logits for each pixel (or equivalently, use a final sigmoid activation to output probabilities).
- **Loss Function:** Use the VAE loss which is the negative ELBO: the sum of reconstruction loss and KL divergence.
 - For reconstruction loss, use binary cross-entropy between the input image and the decoder's output probabilities (this corresponds to $-\mathbb{E}_q[\log p_\theta(x | z)]$).
 - For the KL term, use the closed-form expression for KL between two Gaussians:

$$D_{\text{KL}}(\mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x)) \| \mathcal{N}(0, I)) = \frac{1}{2} \sum_{j=1}^d (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1),$$

where d is the dimensionality of z . (*PyTorch:* `torch.distributions.kl_divergence` or the `Normal` distribution object can also compute this.)

- **Training:** Train your VAE on the MNIST training set for a reasonable number of epochs (e.g., start with 10–20 epochs). Use the Adam optimizer (as a guideline, learning rate $\sim 1e-3$) and minibatch SGD. *Note:* Training on CPU is acceptable since our network is small; if using Colab with GPU, it will simply run faster. Monitor the loss (ELBO) to ensure it decreases.
2. **Analyzing VAE Performance:** After training the VAE, perform the following experiments to illustrate what the model has learned:

- (a) **Latent Space Visualization:** If you set the latent space dimension to 2 (for ease of visualization), plot the latent embeddings of the MNIST test set. Take several test images, run them through the encoder to get $\mu_\phi(x)$ (the mean of $q(z | x)$), and plot these points in the 2D plane, using different colors or markers for different digit labels. (If your latent dimension is higher than 2, you can reduce dimensionality for visualization, e.g., take the first two principal components of the latent means, or use t-SNE on the latent vectors.) This visualization should reveal whether the VAE has organized the latent space meaningfully (e.g., points corresponding to the same digit class cluster together or form continuous manifolds).
- (b) **Reconstruction Quality:** Take a random sample of 5–10 test images and run them through the VAE to obtain reconstructions. Display the original images side-by-side with their reconstructed outputs from the decoder. Visually inspect how well the VAE reconstructs the input – are important features (like the strokes of the digit) preserved? Note any blurriness or distortions in the reconstructions.
- (c) **Generation of New Samples:** Using the decoder, generate new digit images from scratch. Do this by sampling random z vectors from the prior $p(z) = \mathcal{N}(0, I)$ (for example, sample 5 random 2D points if $d = 2$, or 5 random 10-dimensional vectors if $d = 10$, etc.), and feed them into the decoder to produce images. Display these generated images. Because the VAE’s prior is a simple distribution, we expect most samples from it to produce plausible digit images (not just random noise), assuming training succeeded.

For each of these tasks: understand your findings: e.g., Latent space: do the clusters of points for each digit make sense (even if they overlap, is there a continuous progression between digits)? Reconstructions: are there particular digits that the VAE reconstructs better or worse? Generations: do the random samples look like valid handwritten digits, and do they reflect the diversity of the dataset? This analysis will enhance your understanding of how well the VAE learned the data distribution.

3. **Comparing with Doersch’s VAE Implementation:** The tutorial by Carl Doersch (2016) includes an open-source implementation of a VAE for MNIST. Review the description of his implementation in the tutorial and the provided code. On your part, compare your VAE to Doersch’s implementation in the following aspects:

- **Architecture:** What type of neural network architecture did you use vs. what Doersch used? (e.g., both use an MLP? how many hidden units or layers, if known?)
- **Output Distribution and Loss:** What output distribution does Doersch’s VAE assume for MNIST, and what loss function is used accordingly?
(Hint: He treats pixel intensities as probabilities and uses a sigmoid cross-entropy loss, effectively modeling a Bernoulli outcome for each pixel.) State whether this matches what you did in your implementation.
- **Latent Dimensionality:** Comment on the latent space size. Doersch observed that the VAE is “insensitive to the dimensionality of z ” in a broad range, except if z is very small or very large. What latent dimension did you choose, and did you experiment with different sizes? If so, did you notice any difference in reconstruction quality or training difficulty?
- **Results Comparison:** Qualitatively compare the results. For example, Doersch provides sample generated digits from his VAE (which mostly look realistic, with a few “in-between” ambiguous digits noted). How do your generated samples and reconstructions compare? Mention any similar “failure cases” (e.g., digits that look like a mix of two classes) you observed, or if certain digits were hard for your model to generate clearly.